

Currency Converter System

VITyarthi – Build Your Own Project

Project Report

A simple, student-level Python project using three main modules.

1. Introduction

This project is a simple menu-based Currency Converter System. I selected this problem because currency conversion is a small real-world task that can be made easier with a Python program. I kept the project simple enough for me to understand and explain during evaluation.

2. Problem Statement

Doing repeated currency calculations manually can be inconvenient. The proposed system accepts an amount, a source currency and a target currency, then calculates the converted value. It also keeps a local history of previous conversions.

3. Objectives

The objectives are to solve a real-world problem using Python, divide the program into three meaningful modules, use file handling, demonstrate CRUD operations, validate common inputs, maintain conversion history and prepare proper documentation.

4. Target Users

The target users are students, beginners learning Python, travellers checking approximate currency values and users who want a simple local currency calculator.

5. Scope

The system supports conversion, currency viewing, adding, updating and deleting currencies, history viewing/searching/clearing and a simple report. It does not use live exchange rates, online payments or a graphical interface.

6. Functional Requirements

FR1: currency conversion. FR2: view currencies. FR3: add currency. FR4: update a rate. FR5: delete a currency. FR6: store history. FR7: search and clear history. FR8: view and save a report.

7. Non-Functional Requirements

Usability through a simple menu; reliability through local file storage; maintainability through three clearly separated modules; performance suitable for a small local dataset; portability through the Python standard library; and scalability for future features.

8. Technology Used

Python 3 and its standard library are used. The project uses functions, conditions, loops, dictionaries, modules, text-file handling and basic validation. No external package is required to run the application.

9. Currency Data

The initial sample currencies are INR, USD, EUR, GBP, JPY and AUD. Rates represent the INR value of one unit of each currency. These are fixed academic sample rates and are not live market rates.

10. System Architecture

The project has three main modules. `converter.py` handles calculation, `currency_manager.py` handles currency data and CRUD, and `history_manager.py` handles history and reports. `main.py` controls the menu and connects these modules.

11. Module Description

`converter.py` contains the conversion formula. `currency_manager.py` loads and saves rates and performs Create, Read, Update and Delete operations. `history_manager.py` stores conversion records, supports

searching and clearing, and creates the simple report. `main.py` acts as the controller.

12. Conversion Logic

The system first changes the source amount into INR and then changes INR into the target currency. Formula: Amount in INR = Amount × Source Rate. Converted Amount = Amount in INR ÷ Target Rate. This allows many currency pairs without writing a separate formula for each pair.

13. Data Storage

The program uses `currencies.txt` for rates, `history.txt` for conversion history and `conversion_report.txt` for the generated report. Text files were selected because they are simple and suitable for this course-level project.

14. CRUD Operations

Create is adding a currency. Read is viewing currencies and history. Update changes an existing currency rate. Delete removes a currency while protecting INR because it is the base currency.

15. Error Handling

The program checks invalid menu choices, invalid numeric input, zero or negative amounts, unknown currency codes, duplicate currency codes and invalid three-letter codes. The validation is kept simple so the code remains understandable.

16. Testing

Testing covers normal conversions, reverse conversions, invalid inputs, CRUD operations, history search/clear and report saving. The detailed test cases are included in `docs/testing.md`.

17. Design Documentation

The project includes a process-flow diagram, architecture diagram, use-case diagram, component diagram and sequence diagram. These diagrams describe the actual three-module structure rather than adding unnecessary components.

18. GitHub Organization

The repository contains the source code, README, project statement, data files, diagrams, testing document and report. This structure makes the project easier to understand and upload to GitHub.

19. Limitations

The exchange rates are fixed sample values, there is no live API, the application is command-line based and text files are used instead of a database. These choices were made to keep the project simple and explainable.

20. Future Enhancements

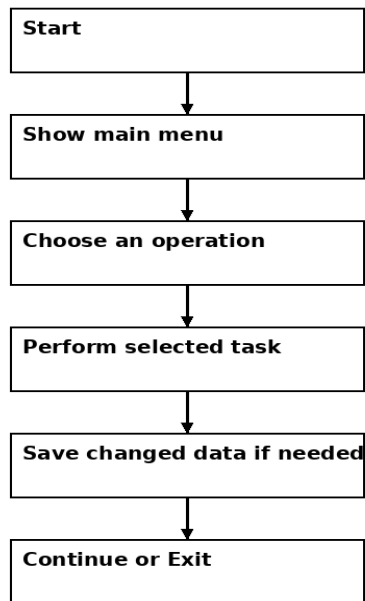
Future versions could use a live exchange-rate API, a graphical interface, a database, user accounts, automated unit tests and more detailed charts.

21. Conclusion

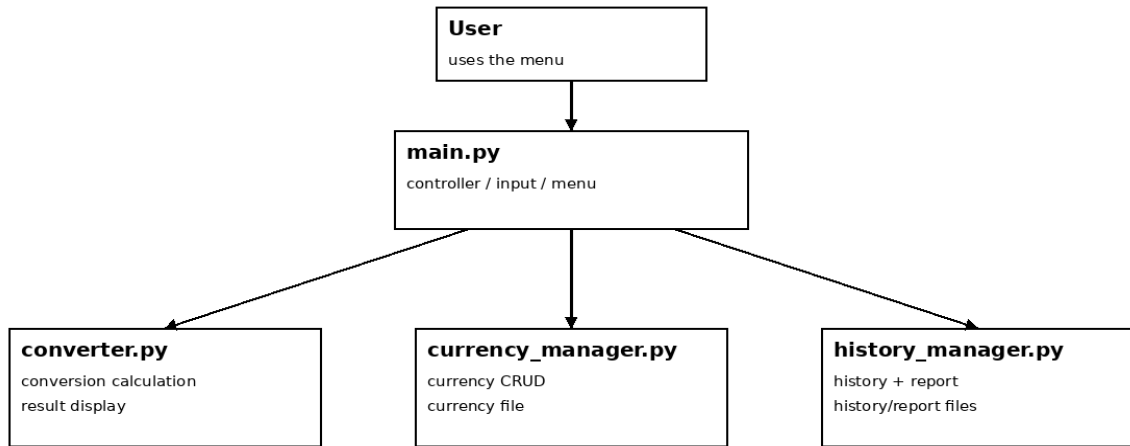
The Currency Converter System shows how a small real-world problem can be solved with a structured Python program. The three-module design keeps the project organized while still being simple enough for a student to understand, demonstrate and explain.

22. Design Diagrams

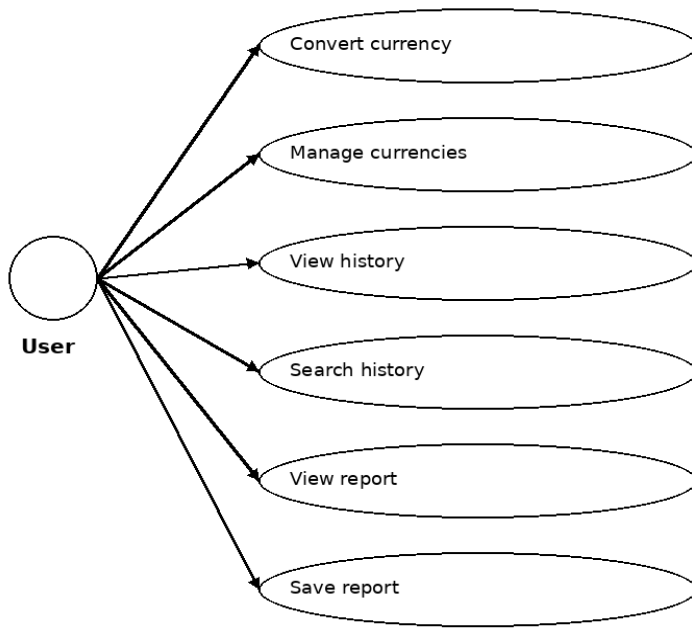
Process Flow



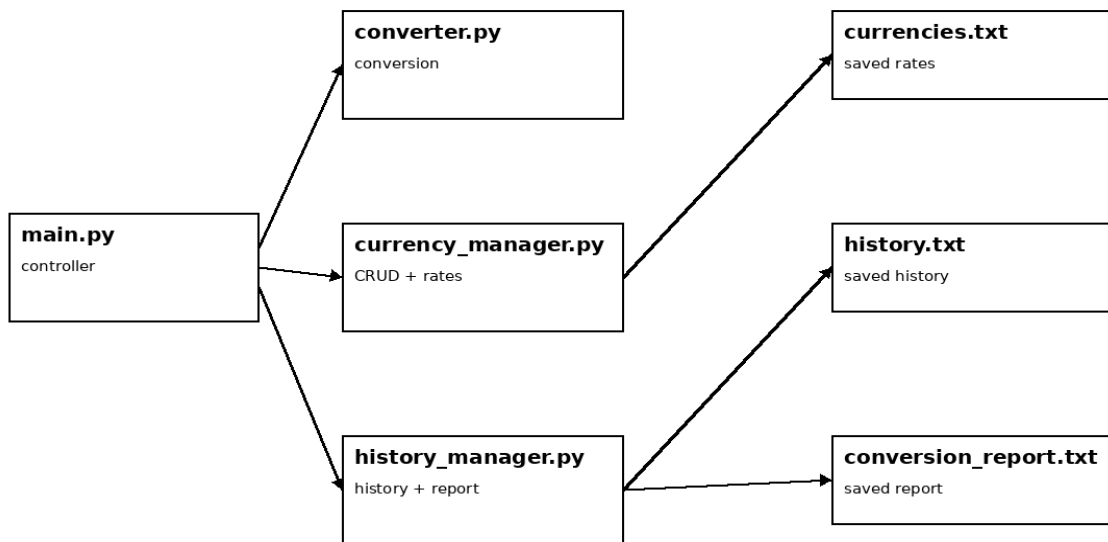
Architecture



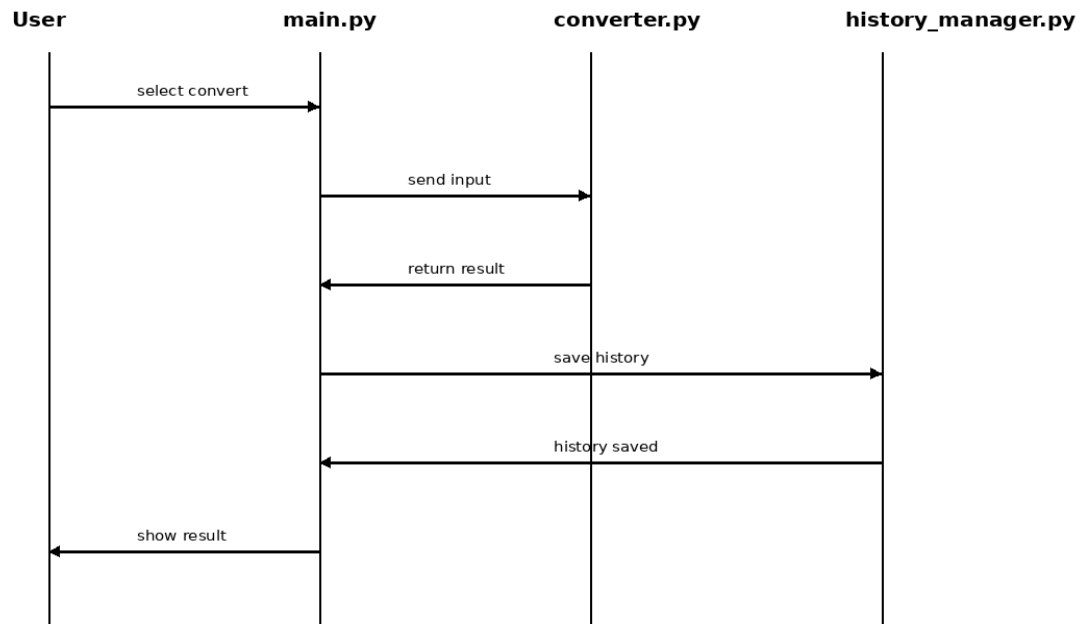
Use Case



Component Diagram



Sequence Diagram



23. Test Case Summary

ID	Test	Expected result
T01	100 USD to INR	8300 INR
T02	1000 INR to JPY	Approx. 1785.71 JPY
T03	Zero amount	Rejected
T04	Text as amount	Valid-number message
T05	Invalid menu choice	Invalid-choice message
T06	Add currency	Currency saved
T07	Duplicate currency	Rejected
T08	Update rate	New rate saved
T09	Delete INR	Rejected
T10	View history	History displayed
T11	Search history	Matching record displayed
T12	Save report	Report file updated

24. Conclusion

The final project keeps the important requirements while avoiding unnecessary complexity. The three modules have clear responsibilities and the project is suitable for a beginner-level VITyarthi demonstration.